1 Background

1.1 Dimensions

The constants in Table 1 define the sizes of the various arrays described later. These are from the `Config struct`.

Math	Code	Explanation
n_L	`n_layers`	number of layers in the network
n_S	`seq_len`	max sequence length
n_D	`dim`	transformer dimension
n_H	`n_heads`	number of query heads
$n_{D\texttt{KV}}$	`kv_dim`	$(n_D \times n_{H\texttt{KV}})/n_H$
XXXX	`hidden_dim`	for ffn layers
n_V	`vocab_size`	vocabulary size
$n_{H\texttt{KV}}$	`n_kv_heads`	number of key/value heads
n_{H_0}	`n_heads * head_size`	
n_{H_1}	`n_kv_heads * head_size`	

Table 1: Dimensions

1.2 Definitions

Definition 1 *A 1D matrix,* `M`*, is defined using a type* `T` *and positive integer* n_X*, representing the dimensions/shape.*

- $\texttt{M}[x]$ *is a scalar of type* `T`
- `M` *is stored as a linear array at address* $m_{\texttt{M}}$ *such that*
    1. *Address of* $\texttt{M}[x] = m_{\texttt{M}} + x$

Definition 2 *A 2D matrix,* `M`*, is defined using a type* `T` *and positive integers* n_X, n_Y*, representing the dimensions/shape.*

- $\texttt{M}[x]$ *is a vector of length* n_Y *and type* `T`
- $\texttt{M}[x, y]$ *is a scalar of type* `T`
- `M` *is stored as a linear array at address* $m_{\texttt{M}}$ *such that*

Name	Shape	Comment	n_X	n_Y	n_Z
`x`	1D array	activation at current timestamp	n_D		
`xb`	1D array	like `x` but inside residual branch	n_D		
`xb2`	1D array	like `xb`	n_D		
`sq`	1D array	n_H			
`sk`	1D array	n_D			
`sk`	1D array	n_D			
`KC`	3D array	keys of kv-cache	n_L	n_S	n_S
`KV`	3D array	values of kv-cache	n_L	n_S	n_S

Table 2: Data structures for state

1. *Address of* $\texttt{M}[x] = m_{\texttt{M}} + (x \times n_Y)$
2. *Address of* $\texttt{M}[x, y] = m_{\texttt{M}} + (x \times n_Y) + y$

Definition 3 *A 3D matrix,* `M`*, is defined using a type* `T` *and positive integers* n_X, n_Y, n_Z*, representing the dimensions/shape.*

- $\texttt{M}[x]$ *is a 2D matrix of dimensions* n_X, n_Y *and type* `T`
- $\texttt{M}[x, y]$ *is a vector of length* n_S *and type* `T`
- $\texttt{M}[x, y, z]$ *is a scalar of type* `T`
- `M` *is stored as a linear array at address* $m_{\texttt{M}}$ *such that*
  1. *Address of* $\texttt{M}[x] = m_{\texttt{M}} + x \times (n_Y \times n_Z)$
  2. *Address of* $\texttt{M}[x, y] = m_{\texttt{M}} + (x \times n_Y \times n_Z) + (y \times n_Z)$
  3. *Address of* $\texttt{M}[x, y, z] = m_{\texttt{M}} + (x \times n_X \times n_Y) + (y \times n_Z) + z$

2 State

The state is represented by the data structures listed in Table 2

3 Weights

The weights are represented by the data structures listed in Table 3

Name	Shape	Comment	n_X	n_Y	n_Z
`tet`	2D array	Lookup table maps token IDs to vectors	n_V	n_D	
`wk`	3D array	key projections	n_L	n_D	n_{H_1}
`wv`	3D array	value projections	n_L	n_D	n_{H_1}

Table 3: Data structures for weights

4 Code

Given

1. an integer p where $0 \leq p < n_P$
2. an integer t where $0 \leq t < n_V$

4.1 rmsnorm

Figure 1: Pseudo-code for `rmsnorm`() function

4.2 vecmatmul

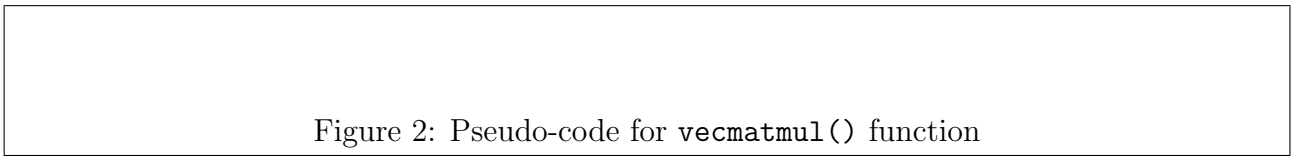

Figure 2: Pseudo-code for `vecmatmul()` function

4.3 forward

```
function foo (
    integer t,
    integer p
)
begin
    foreach l ∈ {0, 1, ... n_L − 1} do
        x ← tet[t]
        xb ← rmsnorm(x, wrms[l])
        q[l] ← vecmatmul(xb, wq[l])
        k[l] ← vecmatmul(xb, wk[l])
        v[l] ← vecmatmul(xb, wv[l])

    endfor
end
```

Figure 3: Pseudo-code for `forward()` function

Function	Section Implemented
`rmsnorm`	Section 1
`vecmatmul`	Section 2
`forward`	Section 3

Table 4: Listing of functions

5 Instructions to LLM

For each of the functions listed in Table 4

1. Write an optimized C function
2. Return a `.c` file — the definition
3. Return a `.h` file — the declaration